PYTHON CASE STUDIES COMPLETE REPORT

Case Study 1: Parking Charge Calculator (15.py)

1. PROBLEM STATEMENT

Design a system to compute parking charges for vehicles (cars or bikes) based on entry and exit hours (0-23 format). Apply hourly rates, dynamic overnight/overflow calculation, surcharges for long stays exceeding 12 hours, and a maximum charge cap of Rs. 300.

2. OBJECTIVES

- Validate input vehicle type and entry/exit time boundaries.
- Calculate actual parking duration across standard and overnight hours.
- Apply specific rates per vehicle type with conditional surcharges and caps.

3. ASSUMPTIONS

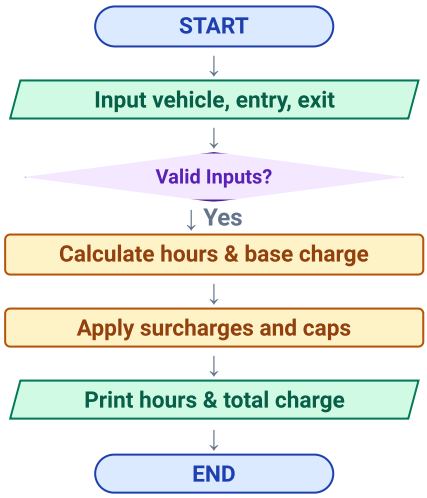
- Hourly rate for Car is Rs. 30 and Bike is Rs. 15.
- If exit hour is less than entry hour, parking crosses midnight (+24 hours).
- Surcharge of Rs. 100 applies if total hours exceed 12; total charge is capped at Rs. 300.

4. ALGORITHM

1. Start.
2. Read `vehicle`, `entry`, and `exit` hour.
3. If time is outside [0, 23], output "Invalid time".
4. Else if vehicle is not "car" or "bike", output "Invalid vehicle type".
5. Else calculate hours: if `exit < entry`, `hours = exit + 24 - entry`, else `hours = exit - entry`.
6. Assign `rate = 30` for car, 15 for bike. Calculate `charge = hours * rate`.
7. If `hours > 12`, add 100 to charge. If `charge > 300`, set `charge = 300`.
8. Display total parking hours and charge. Stop.

Case Study 1 (Continued)

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
  READ vehicle, entry, exit
  IF entry < 0 OR entry > 23 OR exit < 0 OR exit > 23 THEN
    PRINT "Invalid time"
  ELSE IF vehicle != "car" AND vehicle != "bike" THEN
    PRINT "Invalid vehicle type"
  ELSE
    IF exit < entry THEN
      hours = exit + 24 - entry
    ELSE
      hours = exit - entry
    END IF

    IF vehicle == "car" THEN
      rate = 30
    ELSE
      rate = 15
    END IF

    charge = hours * rate
    IF hours > 12 THEN
      charge = charge + 100
    END IF
    IF charge > 300 THEN
      charge = 300
    END IF
    PRINT hours, charge
  END IF
END
```

7. PYTHON SOURCE CODE

```
print("Parking Charge Calculator")

vehicle = input("Enter vehicle type (car/bike): ")
entry = int(input("Enter entry hour (0-23): "))
exit = int(input("Enter exit hour (0-23): "))

if entry < 0 or entry > 23 or exit < 0 or exit > 23:
    print("Invalid time")

elif vehicle != "car" and vehicle != "bike":
    print("Invalid vehicle type")

else:
    if exit < entry:
        hours = exit + 24 - entry
    else:
        hours = exit - entry

    if vehicle == "car":
        rate = 30
    else:
        rate = 15

    charge = hours * rate

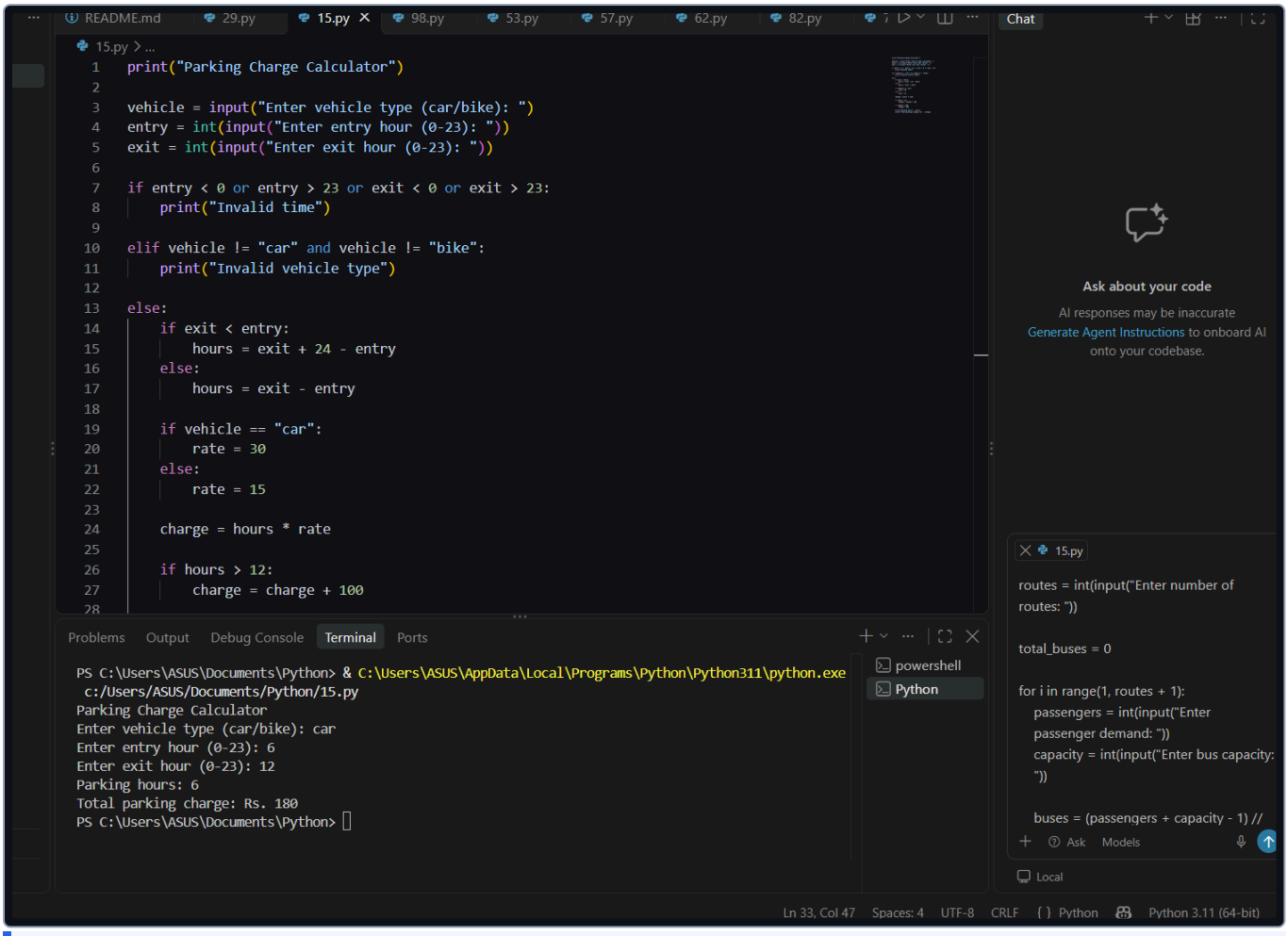
    if hours > 12:
        charge = charge + 100

    if charge > 300:
        charge = 300

    print("Parking hours:", hours)
    print("Total parking charge: Rs.", charge)
```

8. TEST CASES & OUTPUT SCREENSHOTS

Test Case	Input Data	Expected Output
TC-01 (Normal Car)	car, entry: 6, exit: 12	Hours: 6, Charge: Rs. 180



Case Study 2: Password Validation (29.py)

1. PROBLEM STATEMENT

Construct a security module to evaluate candidate passwords against structural policy requirements such as length, character diversity, and forbidden sequence detection.

2. OBJECTIVES

- Enforce minimum character length rules.
- Ensure presence of uppercase, lowercase, numeric digits, and specific special symbols.
- Detect and reject insecure patterns like "123".

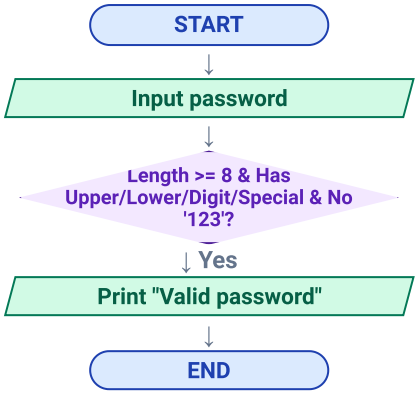
3. ASSUMPTIONS

- Valid special characters are limited to `@`, `#`, `$`, `!`.
- Password checks stop and report at the first failing condition.

4. ALGORITHM

1. Start.
2. Read string input `password`.
3. Check if length < 8; if so, print error.
4. Check for at least one uppercase letter; if missing, print error.
5. Check for at least one lowercase letter; if missing, print error.
6. Check for at least one numeric digit; if missing, print error.
7. Check for special character in "@#$!"; if missing, print error.
8. Check if "123" is in password; if present, print error.
9. If all pass, print "Valid password". Stop.

5. FLOWCHART



Case Study 2 (Continued)

6. PSEUDOCODE

```
BEGIN
  READ password
  IF len(password) < 8 THEN
    PRINT "Password must contain at least 8 characters"
  ELSE IF no uppercase THEN
    PRINT "Password must contain an uppercase letter"
  ELSE IF no lowercase THEN
    PRINT "Password must contain a lowercase letter"
  ELSE IF no digit THEN
    PRINT "Password must contain a digit"
  ELSE IF no special in "@#$!" THEN
    PRINT "Password must contain a special character"
  ELSE IF "123" in password THEN
    PRINT "Password must not contain 123"
  ELSE
    PRINT "Valid password"
  END IF
END
```

7. PYTHON SOURCE CODE

```
print("PASSWORD VALIDATION")

password = input("Enter password: ")

if len(password) < 8:
    print("Password must contain at least 8 characters")

elif not any(ch.isupper() for ch in password):
    print("Password must contain an uppercase letter")

elif not any(ch.islower() for ch in password):
    print("Password must contain a lowercase letter")

elif not any(ch.isdigit() for ch in password):
    print("Password must contain a digit")

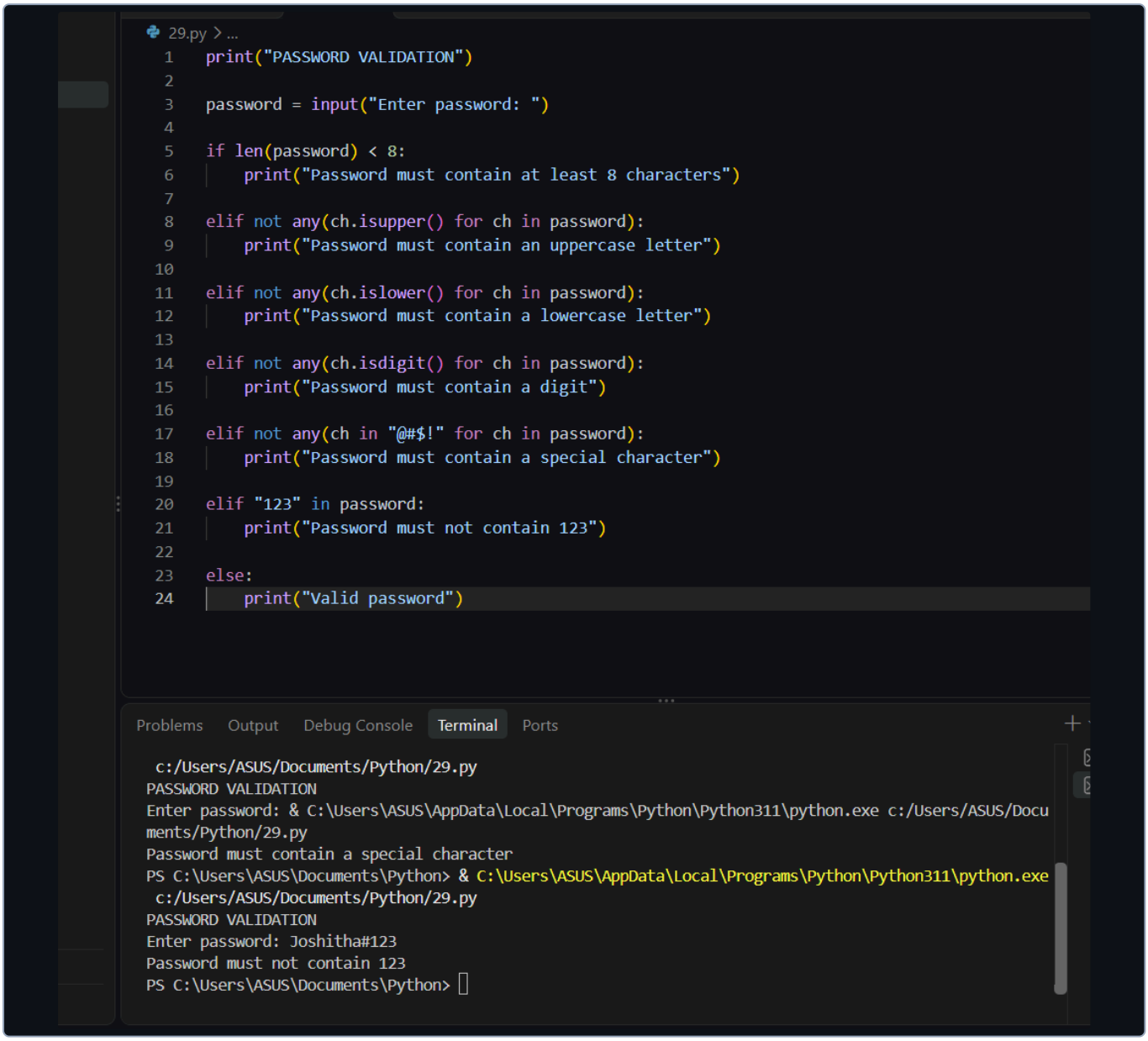
elif not any(ch in "@#$!" for ch in password):
    print("Password must contain a special character")

elif "123" in password:
    print("Password must not contain 123")

else:
    print("Valid password")
```

8. TEST CASES & OUTPUT SCREENSHOTS

Test Case	Input Data	Expected Output
TC-01 (Missing Special)	Joshitha123	"Password must contain a special character"
TC-02 (Forbidden 123)	Joshitha#123	"Password must not contain 123"



Case Study 3: Exam Eligibility Checker (39.py)

1. PROBLEM STATEMENT

Develop an academic eligibility verification system that checks multiple criteria including attendance percentage, laboratory practical completion status, internal examination marks, and fee payment clearance.

2. OBJECTIVES

- Audit student compliance across all academic and administrative requirements.
- Display specific failure reasons for every unmet criterion.
- Grant final exam clearance only when all checks pass.

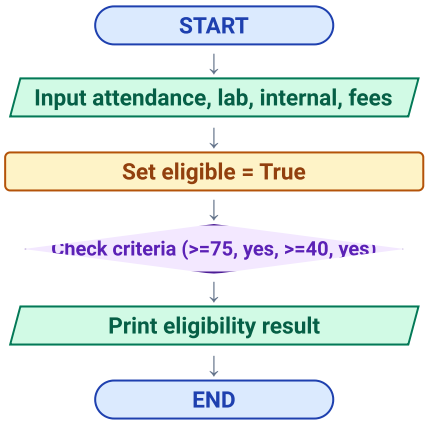
3. ASSUMPTIONS

- Minimum required attendance is 75%.
- Minimum required internal marks score is 40.
- Lab status and Fee status must be explicitly answered as "yes".

4. ALGORITHM

1. Start.
2. Read `attendance`, `lab`, `internal`, and `fees`.
3. Set flag `eligible = True`.
4. If `attendance < 75`, print error and set `eligible = False`.
5. If `lab != "yes"`, print error and set `eligible = False`.
6. If `internal < 40`, print error and set `eligible = False`.
7. If `fees != "yes"`, print error and set `eligible = False`.
8. If `eligible` is True, print final exam clearance; else print rejection message. Stop.

5. FLOWCHART



Case Study 3 (Continued)

6. PSEUDOCODE

```
BEGIN
  READ attendance, lab, internal, fees
  eligible = True
  IF attendance < 75 THEN
    PRINT "Insufficient attendance"
    eligible = False
  END IF
  IF lab != "yes" THEN
    PRINT "Lab not completed"
    eligible = False
  END IF
  IF internal < 40 THEN
    PRINT "Internal marks are insufficient"
    eligible = False
  END IF
  IF fees != "yes" THEN
    PRINT "Fees not cleared"
    eligible = False
  END IF
  IF eligible THEN
    PRINT "Eligible for final examination"
  ELSE
    PRINT "Not eligible for final examination"
  END IF
END
```

7. PYTHON SOURCE CODE

```
attendance = int(input("Enter attendance percentage: "))
lab = input("Lab completed? (yes/no): ")
internal = int(input("Enter internal marks: "))
fees = input("Fees paid? (yes/no): ")

eligible = True

if attendance < 75:
    print("Insufficient attendance")
    eligible = False

if lab != "yes":
    print("Lab not completed")
    eligible = False

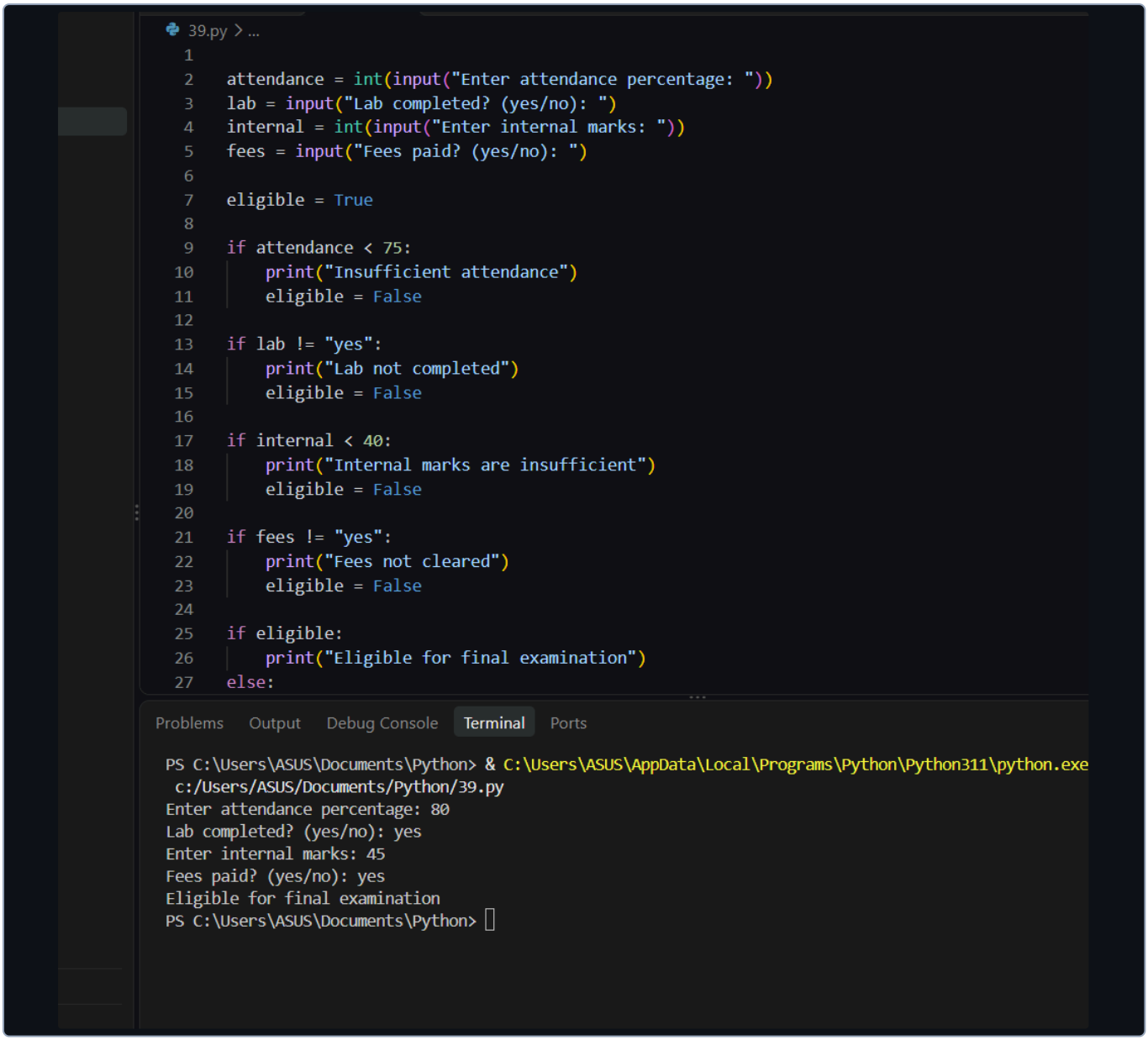
if internal < 40:
    print("Internal marks are insufficient")
    eligible = False

if fees != "yes":
    print("Fees not cleared")
    eligible = False

if eligible:
    print("Eligible for final examination")
else:
    print("Not eligible for final examination")
```

8. TEST CASES & OUTPUT SCREENSHOTS

Test Case	Input Data	Expected Output
TC-01 (All Qualified)	att: 80, lab: yes, int: 45, fees: yes	"Eligible for final examination"



Case Study 4: Tournament Rankings (47.py)

1. PROBLEM STATEMENT

Build a sports tournament standings aggregator that accepts performance statistics for multiple teams, calculates total accumulated points, and outputs the overall ranking list.

2. OBJECTIVES

- Collect structured statistics (wins, draws, goal difference) for `N` teams.
- Compute total points using standard tournament scoring rule equations.
- Format and output ordered team standings.

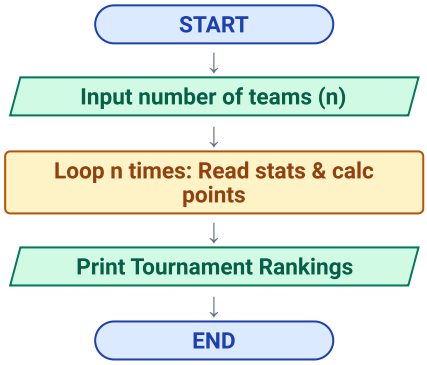
3. ASSUMPTIONS

- Win awards 3 points; Draw awards 1 point; Loss awards 0 points.
- Goal difference serves as secondary tie-breaking tracking criteria.

4. ALGORITHM

1. Start.
2. Read integer n (number of teams).
3. Loop n times: read `name`, `wins`, `draws`, `goals`.
4. Compute `points = wins * 3 + draws`.
5. Store `[name, points, goals]` in list `teams`.
6. Loop through `teams` list and print rank, team name, points, and goal difference. Stop.

5. FLOWCHART



Case Study 4 (Continued)

6. PSEUDOCODE

```
BEGIN
  READ n
  FOR i = 0 TO n-1 DO
    READ name, wins, draws, goals
    points = wins * 3 + draws
    APPEND [name, points, goals] TO teams
  END FOR
  PRINT "Tournament Rankings:"
  FOR i = 0 TO n-1 DO
    PRINT i+1, teams[i][0], teams[i][1], teams[i][2]
  END FOR
END
```

7. PYTHON SOURCE CODE

```
n = int(input("Enter number of teams: "))

teams = []

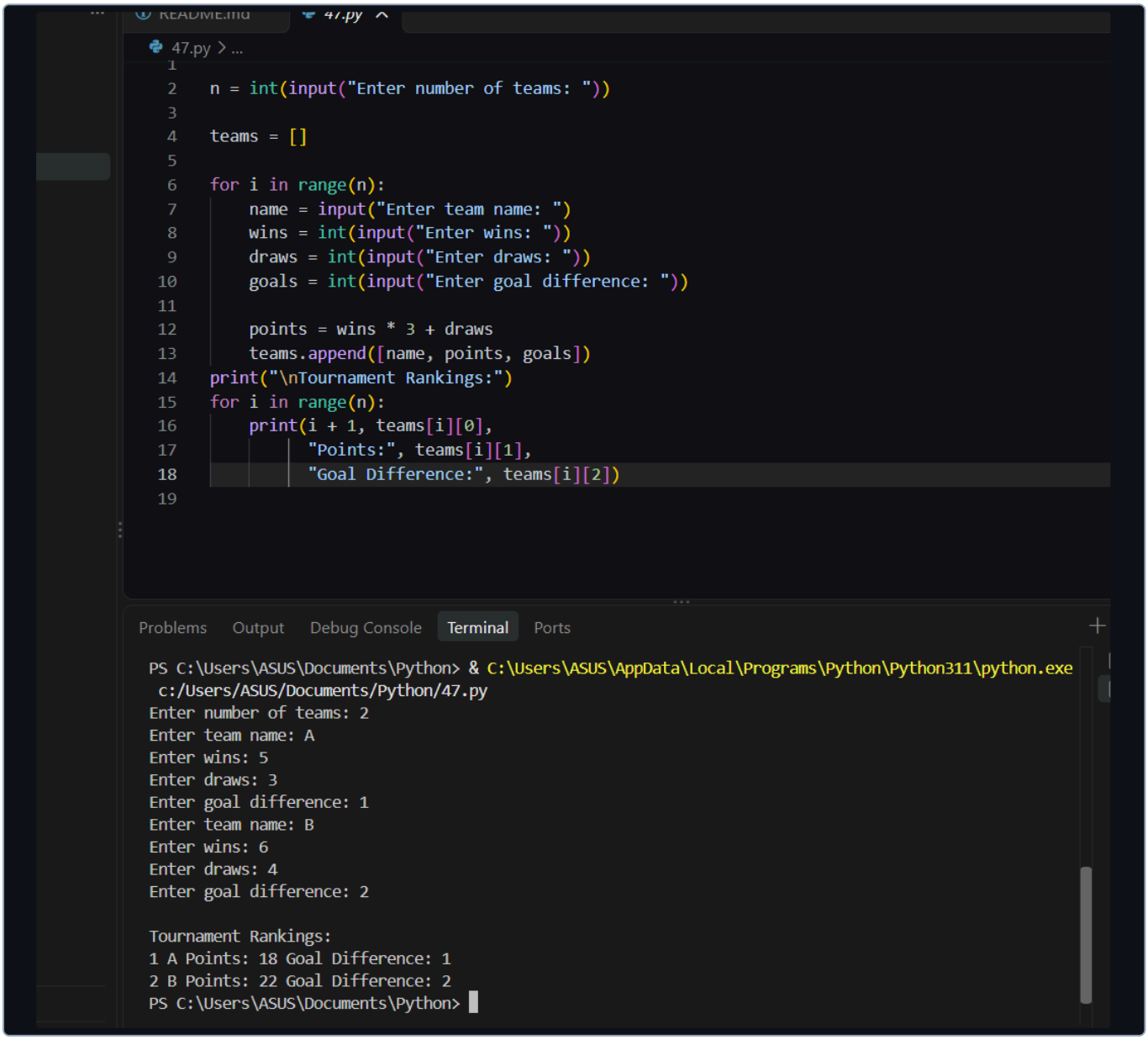
for i in range(n):
    name = input("Enter team name: ")
    wins = int(input("Enter wins: "))
    draws = int(input("Enter draws: "))
    goals = int(input("Enter goal difference: "))

    points = wins * 3 + draws
    teams.append([name, points, goals])

print("\nTournament Rankings:")
for i in range(n):
    print(i + 1, teams[i][0],
          "Points:", teams[i][1],
          "Goal Difference:", teams[i][2])
```

8. TEST CASES & OUTPUT SCREENSHOTS

Test Case	Input Data	Expected Output
TC-01 (2 Teams)	Team A (5W, 3D, 1GD), Team B (6W, 4D, 2GD)	A: 18 Pts, B: 22 Pts



Case Study 5: Food Waste Audit (53.py)

1. PROBLEM STATEMENT

Design an operational monitoring system for institutional kitchens to measure daily meal waste percentages and flag instances exceeding acceptable waste limits.

2. OBJECTIVES

- Calculate leftover meals based on total prepared vs served meals.
- Compute exact waste percentage.
- Evaluate waste percentage against acceptable threshold limits.

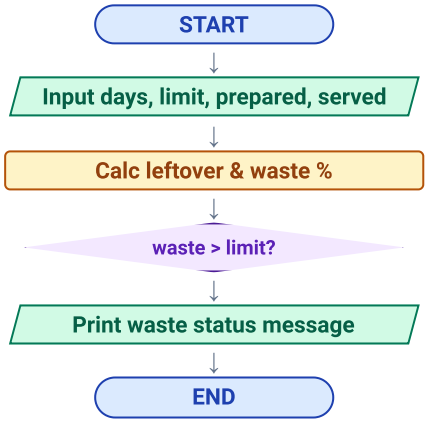
3. ASSUMPTIONS

- Prepared meals count is always greater than 0.
- Waste percentage equation: `(leftover / prepared) * 100`.

4. ALGORITHM

1. Start.
2. Read `days`, `limit` percentage, `prepared` count, and `served` count.
3. Calculate `leftover = prepared - served`.
4. Calculate `waste = (leftover / prepared) * 100`.
5. Print leftover meals count and calculated waste percentage.
6. If `waste > limit`, print "High food waste!"; else print "Food waste is acceptable.". Stop.

5. FLOWCHART



Case Study 5 (Continued)

6. PSEUDOCODE

```
BEGIN
  READ days, limit, prepared, served
  leftover = prepared - served
  waste = (leftover / prepared) * 100
  PRINT leftover, waste
  IF waste > limit THEN
    PRINT "High food waste!"
  ELSE
    PRINT "Food waste is acceptable."
  END IF
END
```

7. PYTHON SOURCE CODE

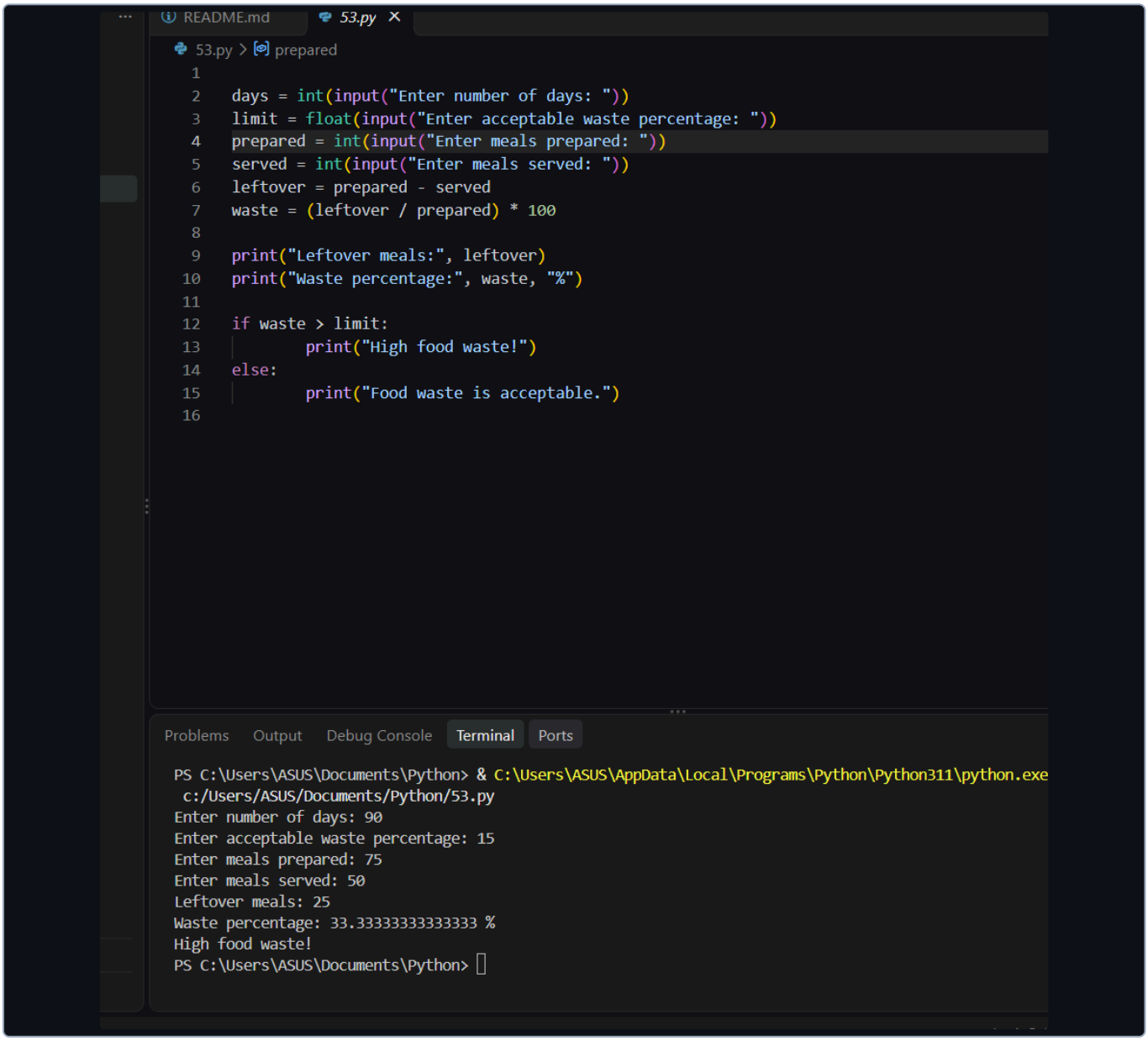
```
days = int(input("Enter number of days: "))
limit = float(input("Enter acceptable waste percentage: "))
prepared = int(input("Enter meals prepared: "))
served = int(input("Enter meals served: "))
leftover = prepared - served
waste = (leftover / prepared) * 100

print("Leftover meals:", leftover)
print("Waste percentage:", waste, "%")

if waste > limit:
    print("High food waste!")
else:
    print("Food waste is acceptable.")
```

8. TEST CASES & OUTPUT SCREENSHOTS

Test Case	Input Data	Expected Output
TC-01 (High Waste)	Limit: 15%, Prep: 75, Served: 50	Waste: 33.33%, "High food waste!"



Case Study 6: Daily Inventory & Sales Management (57.py)

1. PROBLEM STATEMENT

Construct a retail point-of-sale inventory tracker to handle batch order requests, deduct existing stock, accumulate daily revenue, and guard against stock deficits.

2. OBJECTIVES

- Process sequential item orders under initial stock constraints.
- Reject negative quantities and orders exceeding available stock.
- Generate end-of-day sales revenue and remaining stock summaries.

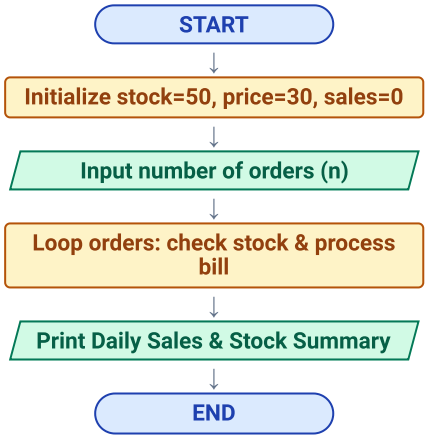
3. ASSUMPTIONS

- Initial stock unit count is 50; fixed price per unit is Rs. 30.

4. ALGORITHM

1. Start. Set `stock = 50`, `price = 30`, `sales = 0`.
2. Read integer n (number of order attempts).
3. Loop n times: read `qty`.
4. If `qty <= 0`, print "Invalid quantity".
5. Else if `qty > stock`, print "Not enough stock".
6. Else calculate `bill = qty * price`, `stock = stock - qty`, `sales = sales + bill`. Print bill amount.
7. After loop, display total revenue sales and remaining stock count. Stop.

5. FLOWCHART



Case Study 6 (Continued)

6. PSEUDOCODE

```
BEGIN
  stock = 50
  price = 30
  sales = 0
  READ n
  FOR i = 0 TO n-1 DO
    READ qty
    IF qty <= 0 THEN
      PRINT "Invalid quantity"
    ELSE IF qty > stock THEN
      PRINT "Not enough stock"
    ELSE
      bill = qty * price
      stock = stock - qty
      sales = sales + bill
      PRINT "Bill amount:", bill
    END IF
  END FOR
  PRINT "Total Sales:", sales
  PRINT "Remaining Stock:", stock
END
```

7. PYTHON SOURCE CODE

```
stock = 50
price = 30
sales = 0

n = int(input("Enter number of orders: "))

for i in range(n):
    qty = int(input("Enter order quantity: "))

    if qty <= 0:
        print("Invalid quantity")

    elif qty > stock:
        print("Not enough stock")

    else:
        bill = qty * price
        stock = stock - qty
        sales = sales + bill

        print("Bill amount:", bill)

print("\nDaily Summary")
print("Total Sales:", sales)
print("Remaining Stock:", stock)
```

8. TEST CASES & OUTPUT SCREENSHOTS

Test Case	Input Data	Expected Output
TC-01 (Multi Orders)	Orders: 3 (qty: 5, 4, 7)	Sales: Rs. 480, Stock left: 34

README.md 57.py

57.py > ...

```python

n = int(input("Enter number of orders: "))

for i in range(n):
    qty = int(input("Enter order quantity: "))

    if qty <= 0:
        print("Invalid quantity")

    elif qty > stock:
        print("Not enough stock")

    else:
        bill = qty * price
        stock = stock - qty
        sales = sales + bill

        print("Bill amount:", bill)

print("\nDaily Summary")
print("Total Sales:", sales)
print("Remaining Stock:", stock)
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\ASUS\Documents\Python> & C:\Users\ASUS\AppData\Local\Programs\Python\Python311\python.exe
 c:/Users/ASUS/Documents/Python/57.py
Enter number of orders: 3
Enter order quantity: 5
Bill amount: 150
Enter order quantity: 4
Bill amount: 120
Enter order quantity: 7
Bill amount: 210

Daily Summary
Total Sales: 480
Remaining Stock: 34
PS C:\Users\ASUS\Documents\Python>
```

Case Study 7: Water Demand & Shortage Tracking (62.py)

1. PROBLEM STATEMENT

Develop a regional municipal water utility program that calculates supply deficits across various sectors and identifies the most underserved geographic area.

2. OBJECTIVES

- Collect demand and supply volumetric metrics for `N` sectors.
- Compute non-negative shortage values per area.
- Identify sector suffering from maximum water shortage.

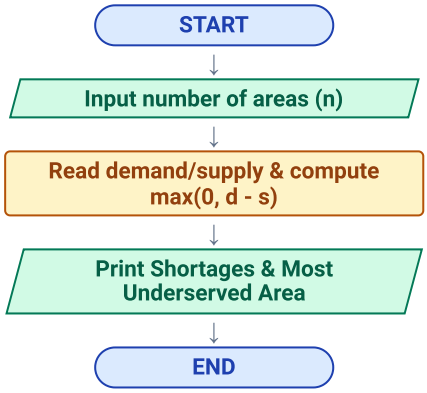
3. ASSUMPTIONS

- Shortage equation: `max(0, demand - supply)`. Excess supply yields zero shortage.

4. ALGORITHM

1. Start.
2. Read integer n (number of areas).
3. Loop n times: read sector `area` name, demand d, supply `s`. Append to lists.
4. Loop n times: compute `shortage = max(0, demand[i] - supply[i])`. Print shortage per area.
5. Find index of maximum shortage and display corresponding underserved area name. Stop.

5. FLOWCHART



Case Study 7 (Continued)

6. PSEUDOCODE

```
BEGIN
  READ n
  FOR i = 0 TO n-1 DO
    READ area, d, s
    APPEND area, d, s TO lists
  END FOR
  FOR i = 0 TO n-1 DO
    shortage = MAX(0, demand[i] - supply[i])
    PRINT area, shortage
  END FOR
  max_idx = INDEX_OF_MAX(shortages)
  PRINT "Most underserved area:", areas[max_idx]
END
```

7. PYTHON SOURCE CODE

```
n = int(input("Enter number of areas: "))

areas = []
demand = []
supply = []

for i in range(n):
    area = input("Enter area name: ")
    d = int(input("Enter water demand: "))
    s = int(input("Enter water supply: "))

    areas.append(area)
    demand.append(d)
    supply.append(s)

shortages = []

for i in range(n):
    shortage = max(0, demand[i] - supply[i])
    shortages.append(shortage)
    print(areas[i], "Shortage:", shortage)

index = shortages.index(max(shortages))
print("Most underserved area:", areas[index])
```

8. TEST CASES & OUTPUT SCREENSHOTS

Test Case	Input Data	Expected Output
TC-01 (2 Sectors)	Area A (D:40, S:35), Area B (D:40, S:45)	A Shortage: 5, Most underserved: A

README.md | 62.py

62.py > ...

```python
areas = []
demand = []
supply = []

for i in range(n):
    area = input("Enter area name: ")
    d = int(input("Enter water demand: "))
    s = int(input("Enter water supply: "))

    areas.append(area)
    demand.append(d)
    supply.append(s)

shortages = []

for i in range(n):
    shortage = max(0, demand[i] - supply[i])
    shortages.append(shortage)
    print(areas[i], "Shortage:", shortage)

index = shortages.index(max(shortages))
print("Most underserved area:", areas[index])
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\ASUS\Documents\Python> & C:\Users\ASUS\AppData\Local\Programs\Python\Python311\python.exe
 c:/Users/ASUS/Documents/Python/62.py
Enter number of areas: 2
Enter area name: A
Enter water demand: 40
Enter water supply: 35
Enter area name: B
Enter water demand: 40
Enter water supply: 45
A Shortage: 5
B Shortage: 0
Most underserved area: A
PS C:\Users\ASUS\Documents\Python>
```

Case Study 8: Tiered Apartment Electricity Billing (77.py)

1. PROBLEM STATEMENT

Design a residential apartment power tariff computer incorporating tiered usage bracket pricing, static maintenance surcharges, low-consumption rebates, and sorted consumption ranking output.

2. OBJECTIVES

- Compute bill per slab: [0-100 units @ Rs.2], (100-200 units @ Rs.3], (>200 units @ Rs.5].
- Include mandatory fixed charge (Rs. 50) and low-use rebate (Rs. 20 for <50 units).
- Sort apartments in descending order of energy units consumed.

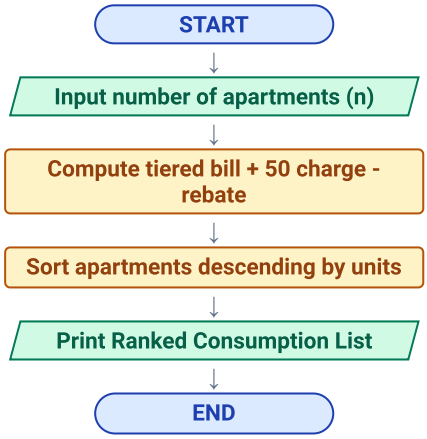
3. ASSUMPTIONS

- Rebate applies only if consumed units strictly fall under 50.

4. ALGORITHM

1. Start. Read integer n (number of apartments).
2. Loop n times: read `name` and `units`.
3. Calculate base bill using slab logic. Add fixed charge 50. If units < 50, subtract rebate 20.
4. Print apartment name and calculated bill. Store in lists.
5. Sort lists in descending order using bubble sort based on units.
6. Print final ranked consumption listing. Stop.

5. FLOWCHART



Case Study 8 (Continued)

6. PSEUDOCODE

```
BEGIN
  READ n
  FOR i = 0 TO n-1 DO
    READ name, units
    IF units <= 100 THEN
      bill = units * 2
    ELSE IF units <= 200 THEN
      bill = 200 + (units - 100) * 3
    ELSE
      bill = 500 + (units - 200) * 5
    END IF
    bill = bill + 50
    IF units < 50 THEN
      bill = bill - 20
    END IF
    PRINT name, bill
  END FOR
  SORT apartments DESCENDING BY units
  PRINT ranked list
END
```

7. PYTHON SOURCE CODE

```
n = int(input("Enter number of apartments: "))

apartments = []
units_list = []

for i in range(n):
    name = input("Enter apartment name: ")
    units = int(input("Enter units consumed: "))

    if units <= 100:
        bill = units * 2
    elif units <= 200:
        bill = 100 * 2 + (units - 100) * 3
    else:
        bill = 100 * 2 + 100 * 3 + (units - 200) * 5

    bill = bill + 50       # Fixed charge

    if units < 50:
        bill = bill - 20    # Rebate

    print(name, "Bill:", bill)

    apartments.append(name)
    units_list.append(units)

print("\nApartments ranked by consumption:")

for i in range(n):
    for j in range(i + 1, n):
        if units_list[i] < units_list[j]:
            units_list[i], units_list[j] = units_list[j], units_list[i]
            apartments[i], apartments[j] = apartments[j], apartments[i]

for i in range(n):
    print(i + 1, apartments[i], "-", units_list[i], "units")
```

8. TEST CASES & OUTPUT SCREENSHOTS

Test Case	Input Data	Expected Output
TC-01 (3 Units)	First: 500, Second: 444, Third: 200	First Bill: 2050, Ranked 1st

README.md 77.py

77.py > ...

```python

n = int(input("Enter number of apartments: "))

apartments = []
units_list = []

for i in range(n):
    name = input("Enter apartment name: ")
    units = int(input("Enter units consumed: "))

    if units <= 100:
        bill = units * 2
    elif units <= 200:
        bill = 100 * 2 + (units - 100) * 3
    else:
        bill = 100 * 2 + 100 * 3 + (units - 200) * 5

    bill = bill + 50       # Fixed charge

    if units < 50:
        bill = bill - 20    # Rebate
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\ASUS\Documents\Python> & C:\Users\ASUS\AppData\Local\Programs\Python\Python311\python.exe
 c:/Users/ASUS/Documents/Python/77.py
Enter number of apartments: 3
Enter apartment name: First
Enter units consumed: 500
First Bill: 2050
Enter apartment name: Second
Enter units consumed: 444
Second Bill: 1770
Enter apartment name: Third
Enter units consumed: 200
Third Bill: 550

Apartments ranked by consumption:
1 First - 500 units
2 Second - 444 units
3 Third - 200 units
PS C:\Users\ASUS\Documents\Python>
```

Case Study 9: Candidate Selection Screening (82.py)

1. PROBLEM STATEMENT

Construct an automated recruitment shortlist generator that screens applicants based on composite eligibility thresholds across academic marks, skill assessment scores, and attendance percentage.

2. OBJECTIVES

- Enforce simultaneous eligibility conditions (`marks >= 60`, `score >= 50`, `attendance >= 75`).
- Filter and reject non-qualifying candidates immediately.
- Print structured summary table of qualified shortlisted candidates.

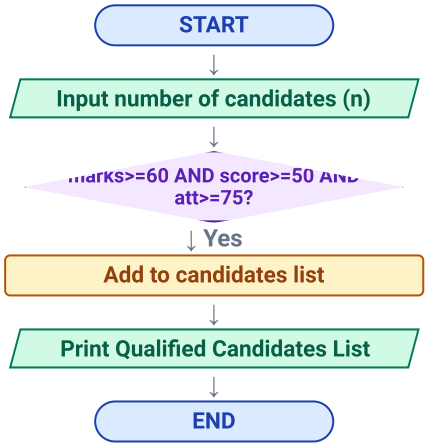
3. ASSUMPTIONS

- All candidate input evaluation scores are integers between 0 and 100.

4. ALGORITHM

1. Start. Read integer n (number of candidate applicants).
2. Loop n times: read `name`, `marks`, `score`, and `attendance`.
3. If `marks >= 60 AND score >= 50 AND attendance >= 75`, append candidate to `candidates` list.
4. Else print candidate name + "is not eligible".
5. After reading all inputs, display "Qualified Candidates Ranking" list. Stop.

5. FLOWCHART



Case Study 9 (Continued)

6. PSEUDOCODE

```
BEGIN
  READ n
  FOR i = 0 TO n-1 DO
    READ name, marks, score, attendance
    IF marks >= 60 AND score >= 50 AND attendance >= 75 THEN
      APPEND [name, marks, score] TO candidates
    ELSE
      PRINT name, "is not eligible"
    END IF
  END FOR
  PRINT "Qualified Candidates Ranking:"
  FOR i = 0 TO len(candidates)-1 DO
    PRINT i+1, candidates[i][0], candidates[i][1], candidates[i][2]
  END FOR
END
```

7. PYTHON SOURCE CODE

```
n = int(input("Enter number of candidates: "))

candidates = []

for i in range(n):
    name = input("Enter candidate name: ")
    marks = int(input("Enter marks: "))
    score = int(input("Enter assessment score: "))
    attendance = int(input("Enter attendance percentage: "))

    if marks >= 60 and score >= 50 and attendance >= 75:
        candidates.append([name, marks, score])
    else:
        print(name, "is not eligible")

print("\nQualified Candidates Ranking:")

for i in range(len(candidates)):
    print(i + 1, candidates[i][0],
          "Marks:", candidates[i][1],
          "Score:", candidates[i][2])
```

8. TEST CASES & OUTPUT SCREENSHOTS

Test Case	Input Data	Expected Output
TC-01 (2 Applicants)	Ram (75, 60, 89), Sita (89, 56, 99)	Both candidates qualify & ranked

README.md 82.py

82.py > ...

```python
n = int(input("Enter number of candidates: "))

candidates = []

for i in range(n):
    name = input("Enter candidate name: ")
    marks = int(input("Enter marks: "))
    score = int(input("Enter assessment score: "))
    attendance = int(input("Enter attendance percentage: "))

    if marks >= 60 and score >= 50 and attendance >= 75:
        candidates.append([name, marks, score])
    else:
        print(name, "is not eligible")
print("\nQualified Candidates Ranking:")

for i in range(len(candidates)):
    print(i + 1, candidates[i][0],
          "Marks:", candidates[i][1],
          "Score:", candidates[i][2])

```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\ASUS\Documents\Python> & C:\Users\ASUS\AppData\Local\Programs\Python\Python311\python.exe
 c:/Users/ASUS/Documents/Python/82.py
Enter number of candidates: 2
Enter candidate name: Ram
Enter marks: 75
Enter assessment score: 60
Enter attendance percentage: 89
Enter candidate name: Sita
Enter marks: 89
Enter assessment score: 56
Enter attendance percentage: 99

Qualified Candidates Ranking:
1 Ram Marks: 75 Score: 60
2 Sita Marks: 89 Score: 56
```

Case Study 10: Semester Progression Audit (98.py)

1. PROBLEM STATEMENT

Build a university degree progression validation engine that checks academic marks, course attendance percentage, subject pass completion counts, and pre-requisite course status.

2. OBJECTIVES

- Validate boundary constraints of input data range metrics.
- Audit rules: Minimum 40 marks, 75% attendance, 100% subjects passed, and prerequisite completion.
- Display detailed outstanding requirements for any failed metric.

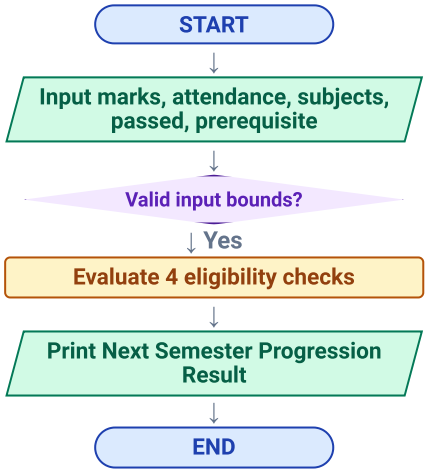
3. ASSUMPTIONS

- Valid marks and attendance percentages range strictly between 0 and 100.

4. ALGORITHM

1. Start. Read `marks`, `attendance`, total `subjects`, `passed` count, and `prerequisite` status.
2. If any inputs are out of valid bounds, print appropriate input error message.
3. Else set `eligible = True`.
4. If `marks < 40`, print requirement error and set `eligible = False`.
5. If `attendance < 75`, print requirement error and set `eligible = False`.
6. If `passed < subjects`, print requirement error and set `eligible = False`.
7. If `prerequisite != "yes"`, print requirement error and set `eligible = False`.
8. If `eligible` is True, print promotion clearance; else print rejection. Stop.

5. FLOWCHART



Case Study 10 (Continued)

6. PSEUDOCODE

```
BEGIN
  READ marks, attendance, subjects, passed, prerequisite
  IF invalid inputs THEN
    PRINT "Invalid inputs"
  ELSE
    eligible = True
    IF marks < 40 THEN
      PRINT "Min 40 marks needed"
      eligible = False
    END IF
    IF attendance < 75 THEN
      PRINT "Min 75% attendance needed"
      eligible = False
    END IF
    IF passed < subjects THEN
      PRINT "Pass all subjects"
      eligible = False
    END IF
    IF prerequisite != "yes" THEN
      PRINT "Complete prerequisites"
      eligible = False
    END IF
    IF eligible THEN
      PRINT "Eligible for next semester"
    ELSE
      PRINT "Not eligible for next semester"
    END IF
  END IF
END
```

7. PYTHON SOURCE CODE

```
print("Semester Progression Eligibility")

marks = int(input("Enter semester marks: "))
attendance = int(input("Enter attendance percentage: "))
subjects = int(input("Enter number of subjects: "))
passed = int(input("Enter number of subjects passed: "))
prerequisite = input("Are prerequisites completed? (yes/no): ").lower()

if marks < 0 or marks > 100:
    print("Invalid marks")
elif attendance < 0 or attendance > 100:
    print("Invalid attendance")
elif subjects <= 0 or passed < 0 or passed > subjects:
    print("Invalid subject details")
else:
    eligible = True

    if marks < 40:
        print("Outstanding requirement: Minimum 40 marks needed")
        eligible = False

    if attendance < 75:
        print("Outstanding requirement: Minimum 75% attendance needed")
        eligible = False

    if passed < subjects:
        print("Outstanding requirement: Pass all subjects")
        eligible = False

    if prerequisite != "yes":
        print("Outstanding requirement: Complete prerequisites")
        eligible = False

    if eligible:
        print("Eligible for next semester")
    else:
        print("Not eligible for next semester")
```

8. TEST CASES & OUTPUT SCREENSHOTS

Test Case	Input Data	Expected Output
TC-01 (All Qualified)	Marks: 85, Att: 90%, Subs: 4, Passed: 4, Pre: yes	"Eligible for next semester"

README.md 98.py

98.py > ...

```python
passed = int(input("Enter number of subjects passed: "))
prerequisite = input("Are prerequisites completed? (yes/no): ").lower()

if marks < 0 or marks > 100:
    print("Invalid marks")
elif attendance < 0 or attendance > 100:
    print("Invalid attendance")
elif subjects <= 0 or passed < 0 or passed > subjects:
    print("Invalid subject details")
else:
    eligible = True

    if marks < 40:
        print("Outstanding requirement: Minimum 40 marks needed")
        eligible = False

    if attendance < 75:
        print("Outstanding requirement: Minimum 75% attendance needed")
        eligible = False

    if passed < subjects:
        print("Outstanding requirement: Pass all subjects")
        eligible = False

    if prerequisite != "yes":
        print("Outstanding requirement: Complete prerequisites")
        eligible = False

    if eligible:
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\ASUS\Documents\Python> & C:\Users\ASUS\AppData\Local\Programs\Python\Python311\python.exe
 c:/Users/ASUS/Documents/Python/98.py
Semester Progression Eligibility
Enter semester marks: 85
Enter attendance percentage: 90
Enter number of subjects: 4
Enter number of subjects passed: 4
Are prerequisites completed? (yes/no): yes
Eligible for next semester
PS C:\Users\ASUS\Documents\Python>
```